

INGENIERÍA DE SOFTWARE II

Práctica de laboratorio: Inspección de código con PMD

Estudiante: Alejandro Tapia

Fecha: 1 de octubre de 2026

Repositorio: <https://github.com/alejandrotapia20/CodeInspection>

1. Objetivo

Usar PMD como herramienta de métrica de calidad para detectar de forma preventiva defectos en el código de un proyecto Java, desde Eclipse y desde Maven, y obtener un informe HTML antes y después de corregir los problemas encontrados.

2. Entorno de trabajo

Herramienta	Versión usada	Versión pedida
Eclipse IDE for Java Developers	2026-09 (4.41.0)	≥ 2024-09 (4.34)
PMD Plug-in para Eclipse	7.28.0 (motor PMD 7.28.0)	Última disponible
maven-pmd-plugin	3.21.2 (motor PMD 6.55.0)	3.21.2
Apache Maven	3.9.16	Integrado (M2E)
JDK	OpenJDK 26.0.1 (compila con release 21)	—

3. Parte 1: Maven en Eclipse

Eclipse 2026-09 ya incluye M2E. Se verificó en Eclipse → About Eclipse IDE → Installation Details → Plug-ins, filtrando por m2e.

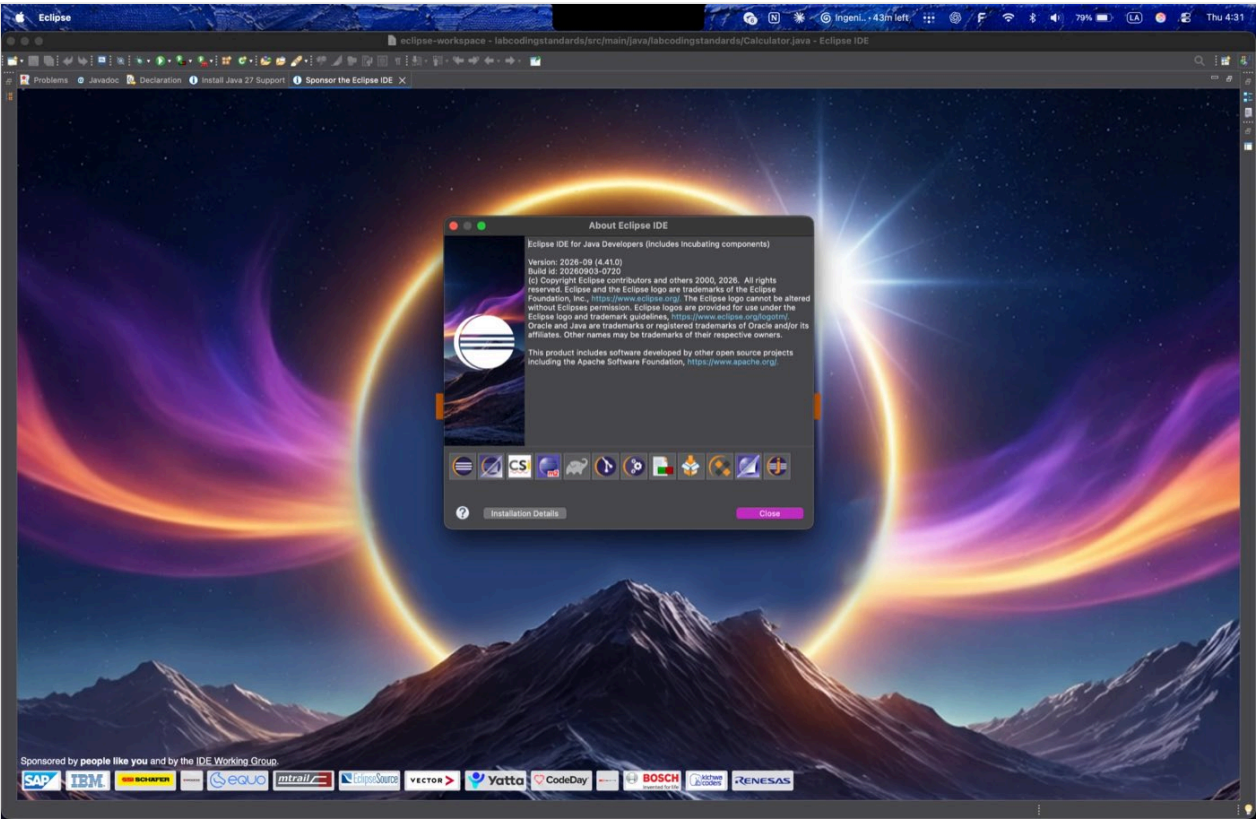


Figura 1. Versión de Eclipse instalada (2026-09).

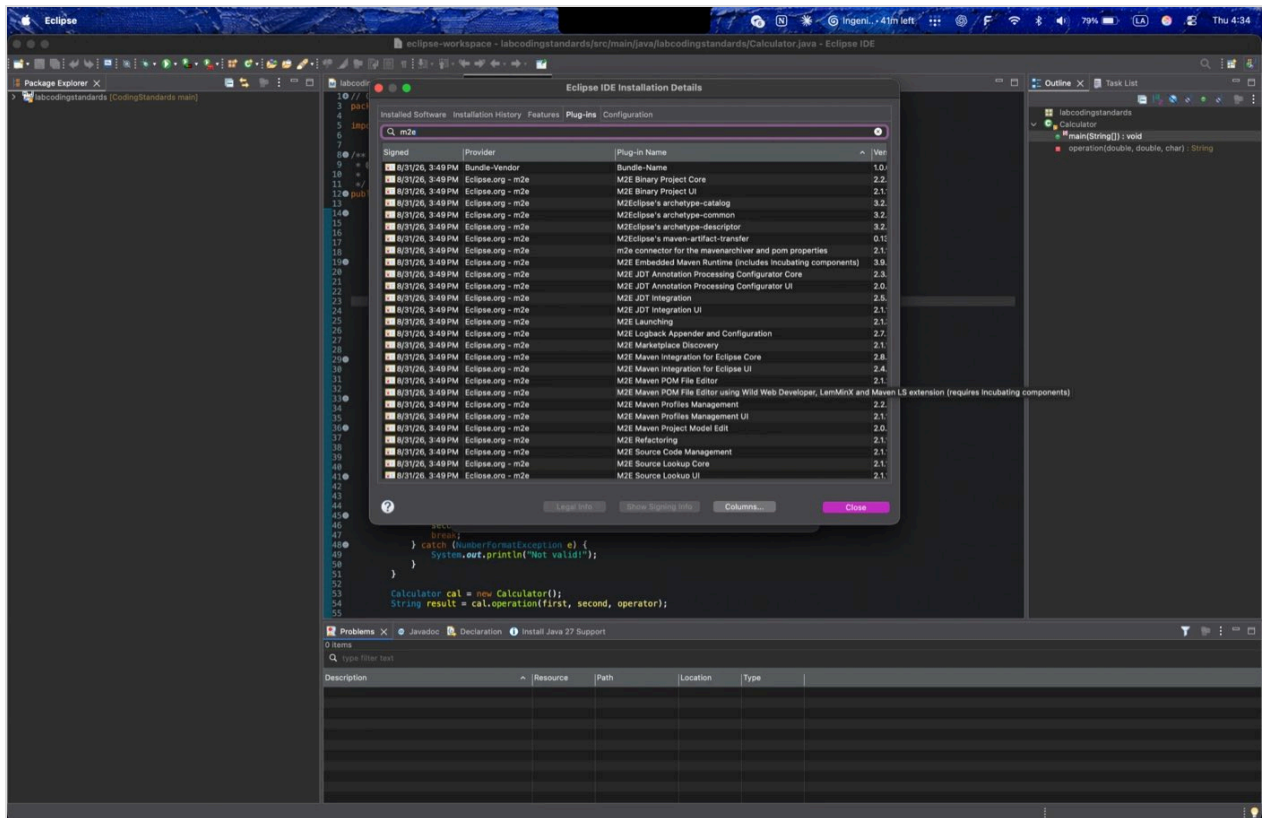


Figura 2. Plug-ins de Maven Integration for Eclipse (M2E).

4. Parte 2: Plugin PMD para Eclipse

Desde Help → Install New Software se agregó el sitio <https://pmd.github.io/pmd-eclipse-plugin-p2-site/> y se instaló PMD for Eclipse. Eclipse pidió confirmar la confianza en la autoridad pmd.github.io, que es el sitio oficial del proyecto.

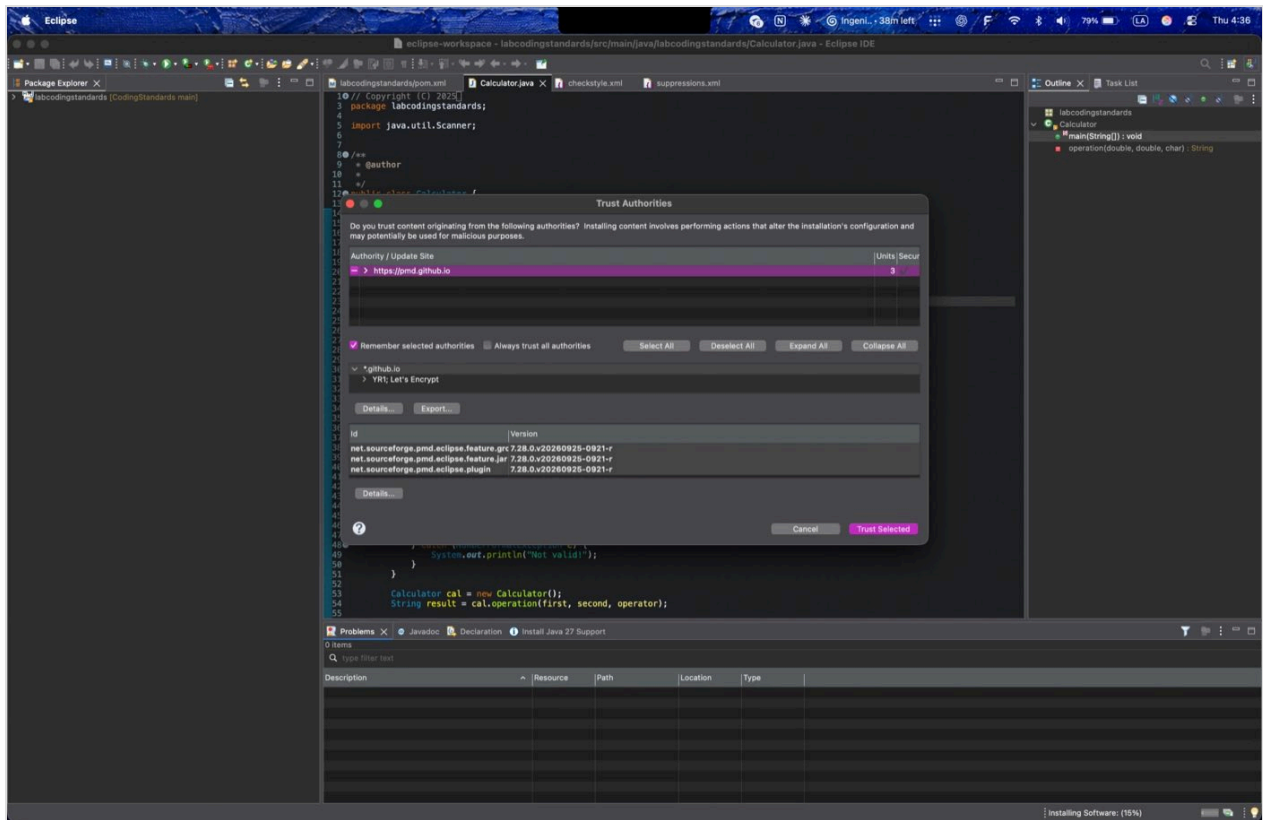


Figura 3. Confirmación de confianza durante la instalación del plugin PMD 7.28.0.

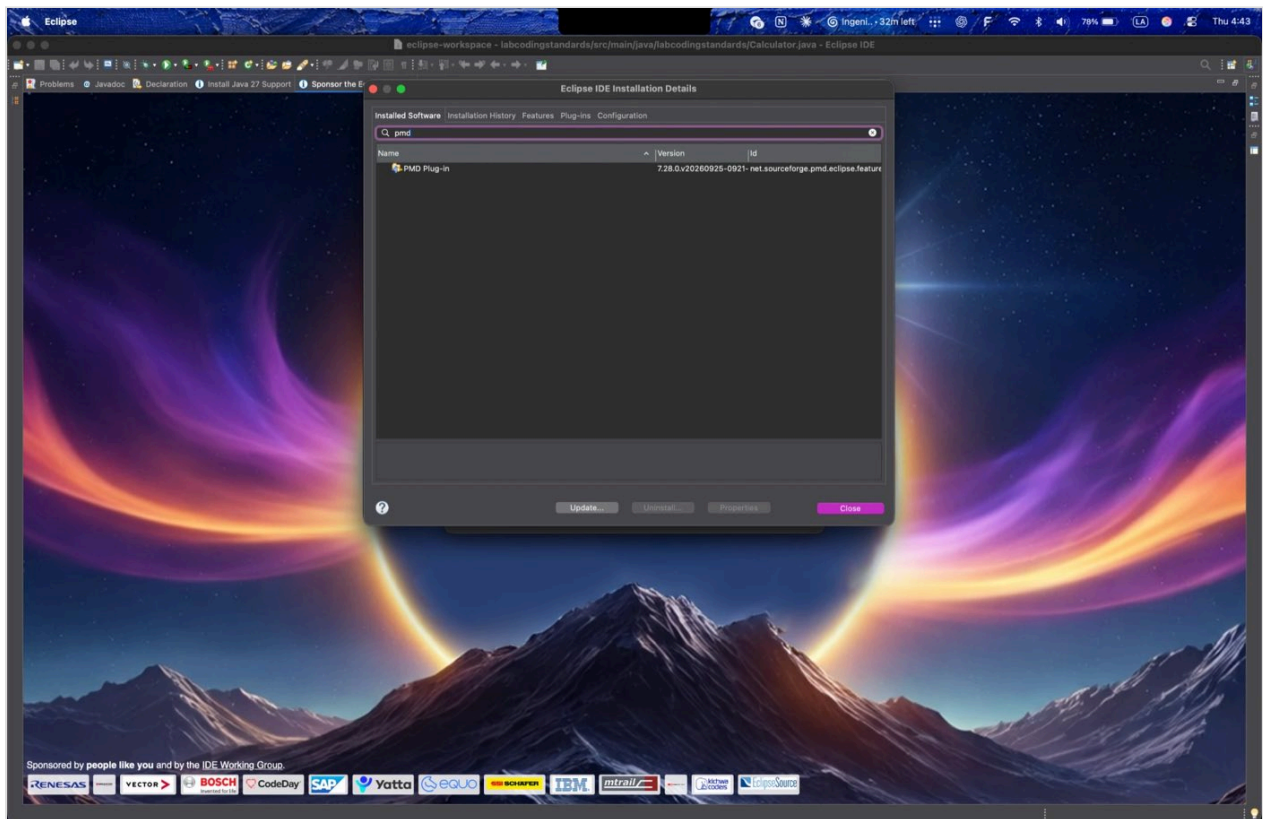


Figura 4. PMD Plug-in instalado.

5. Parte 3: Descargar y configurar el proyecto

Se hizo un fork de leortyz/CodeInspection a la cuenta propia y se clonó localmente. El proyecto venía configurado para Java 1.5, así que se agregó al `pom.xml` la propiedad `maven.compiler.release` con valor 21. También se agregó un `.gitignore` para no versionar la carpeta `target`. Luego se importó con `File → Import → Existing Maven Projects` y se habilitó PMD en las propiedades del proyecto, usando el conjunto de reglas propio `my_ruleset.xml`.

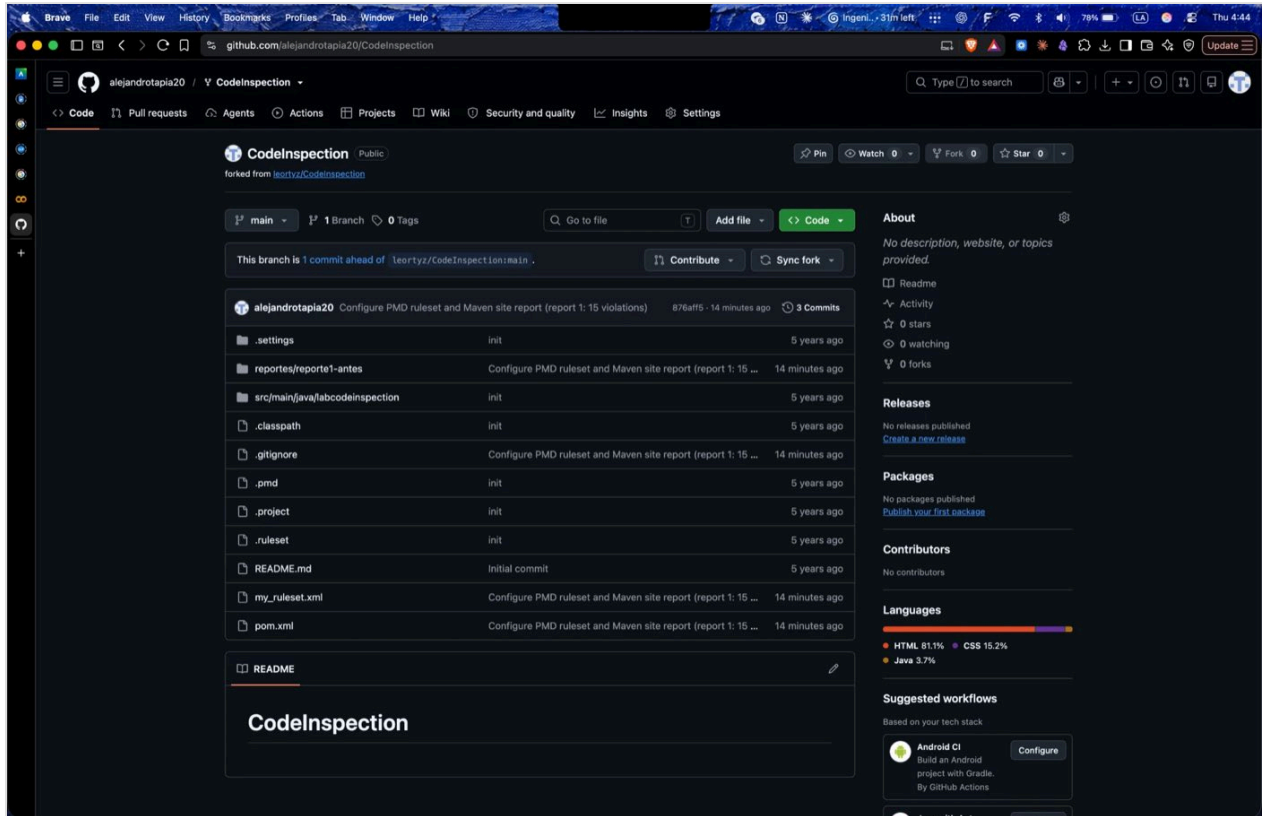


Figura 5. Fork del repositorio en GitHub.

6. Parte 4: Uso de PMD

En Eclipse → Settings → PMD se activó la opción Check code after saving. Después se ejecutó PMD → Check Code sobre el proyecto. El plugin marcó 14 violaciones: 10 en Email.java y 4 en EmailApp.java.

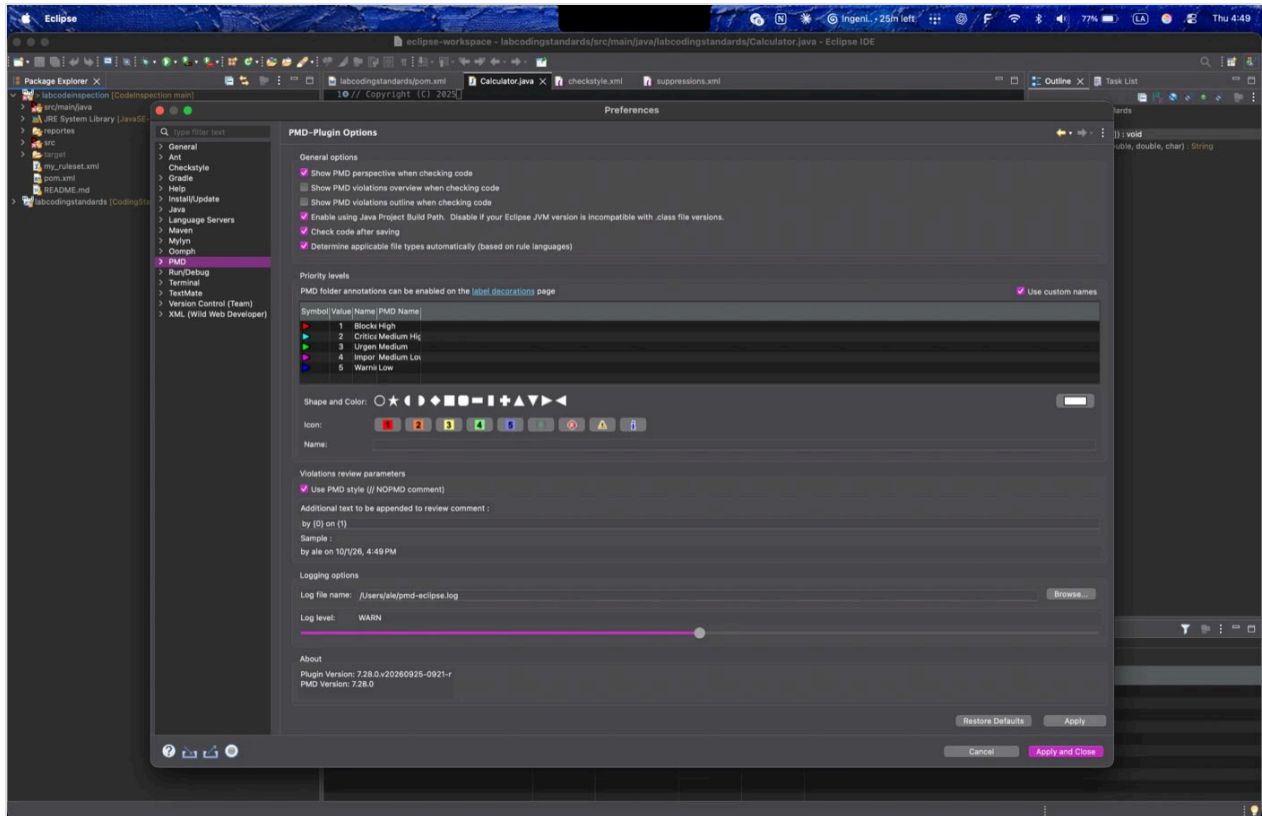


Figura 8. Opción Check code after saving activada.

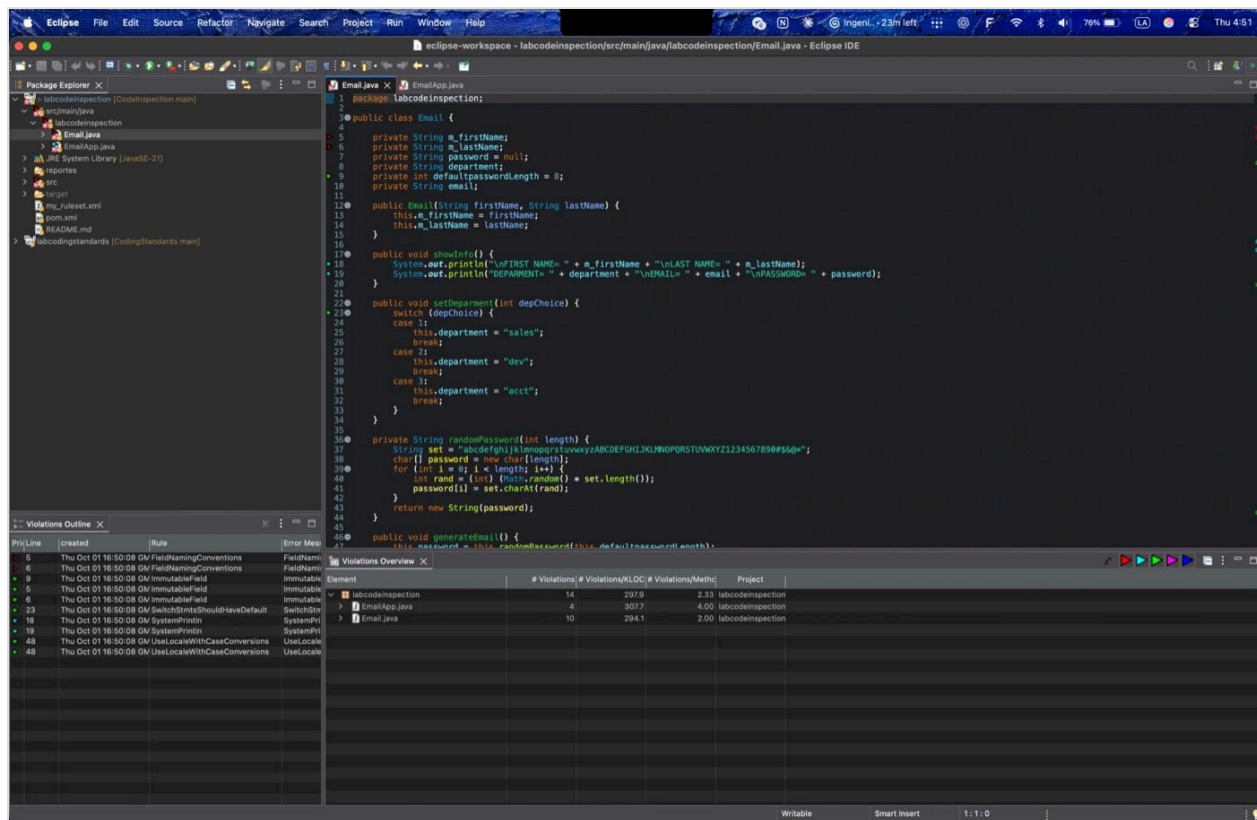


Figura 9. Violaciones detectadas por PMD en Eclipse sobre Email.java.

7. Parte 5: Conjunto de reglas PMD

Se creó `my_ruleset.xml` con siete reglas de cuatro categorías, elegidas porque detectan problemas reales del código del laboratorio:

Categoría	Regla	Qué detecta
Best Practices	SystemPrintln	Uso de System.out en lugar de un logger.
Best Practices	SwitchStmtsShouldHaveDefault	switch sin caso default.
Code Style	FieldNamingConventions	Nombres de atributos fuera de camelCase.
Code Style	ShortVariable	Variables con nombres demasiado cortos.
Design	ImmutableField	Atributos que deberían ser final.
Design	UseUtilityClass	Clases solo con métodos estáticos y constructor público.
Error Prone	UseLocaleWithCaseConversions	toLowerCase/toUpperCase sin Locale.

```

<?xml version="1.0" encoding="UTF-8"?>
<ruleset name="Custom Rules"
  xmlns="https://pmd.github.io/ruleSet/2.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://pmd.github.io/ruleSet/2.0.0
https://pmd.github.io/ruleSet_2_0_0.xsd">
  <description>Code Inspection Lab - Alejandro Tapia</description>

  <!-- Buenas prácticas -->
  <rule ref="category/java/bestpractices.xml/SystemPrintln"/>
  <rule ref="category/java/bestpractices.xml/SwitchStmtsShouldHaveDefault"/>

  <!-- Estilo de código -->
  <rule ref="category/java/codestyle.xml/FieldNamingConventions"/>
  <rule ref="category/java/codestyle.xml/ShortVariable"/>

  <!-- Diseño -->
  <rule ref="category/java/design.xml/ImmutableField"/>
  <rule ref="category/java/design.xml/UseUtilityClass"/>

  <!-- Propensos a errores -->
  <rule ref="category/java/errorprone.xml/UseLocaleWithCaseConversions"/>
</ruleset>

```

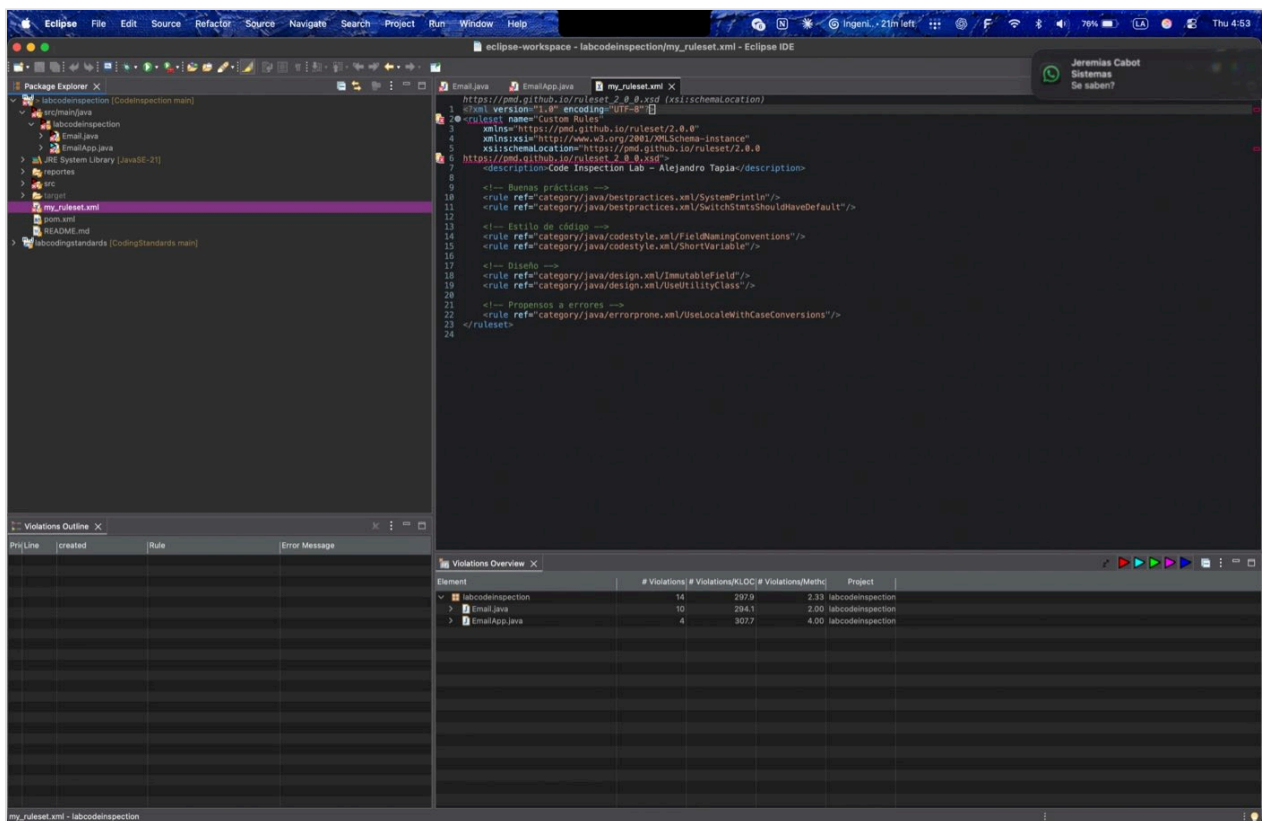


Figura 10. my_ruleset.xml abierto en Eclipse.

8. Parte 6: Reportes PMD con Maven

Se actualizó el pom.xml con los plugins indicados en la guía. Como el proyecto compila con Java 21 y el maven-pmd-plugin 3.21.2 usa PMD 6.55.0, se agregó targetJdk 17 para que PMD analice el código con un nivel de lenguaje que soporta.

```
<build>
  <pluginManagement>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-site-plugin</artifactId>
        <version>3.12.1</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-project-info-reports-plugin</artifactId>
        <version>3.6.2</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-pmd-plugin</artifactId>
        <version>3.21.2</version>
      </plugin>
    </plugins>
  </pluginManagement>
</build>

<reporting>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-pmd-plugin</artifactId>
      <version>3.21.2</version>
      <configuration>
        <rulesets>
          <ruleset>${basedir}/my_ruleset.xml</ruleset>
        </rulesets>
        <targetJdk>17</targetJdk>
      </configuration>
    </plugin>
  </plugins>
</reporting>
```

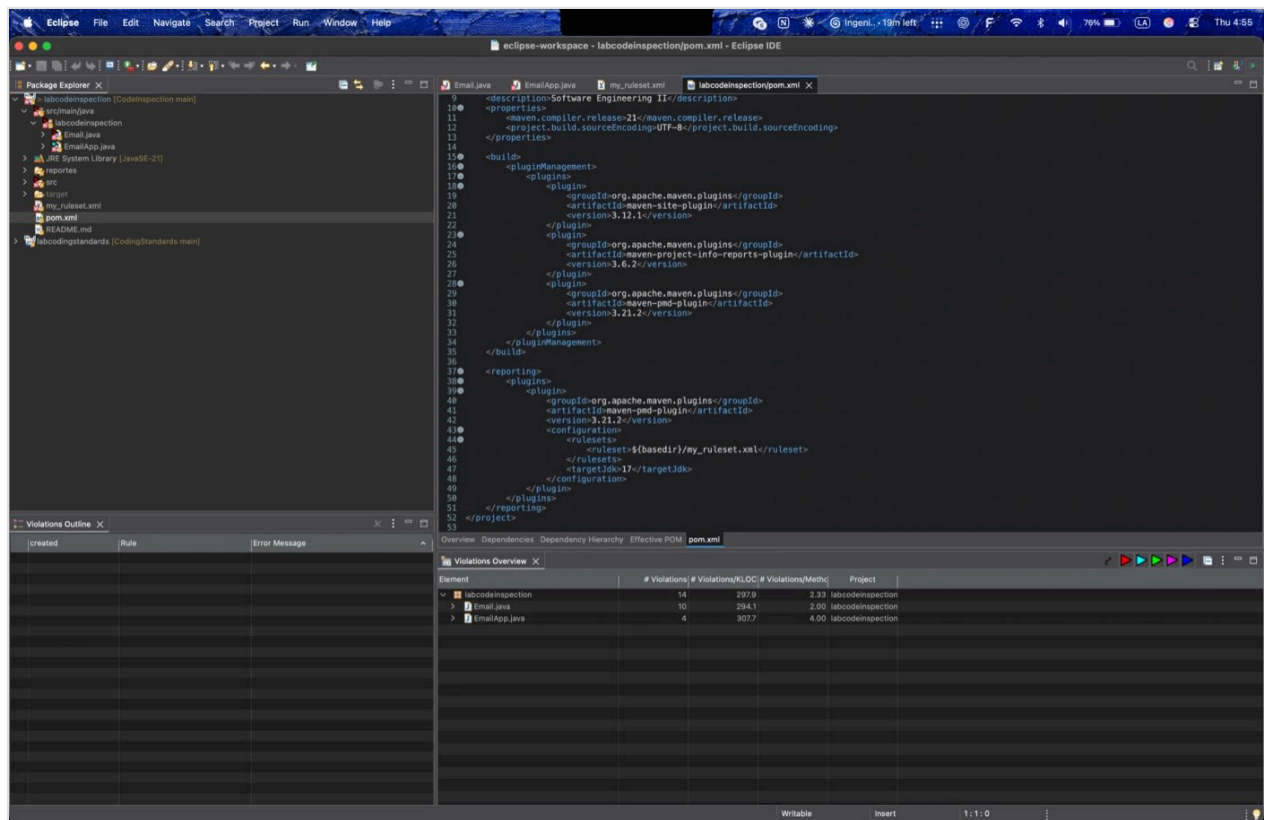


Figura 11. pom.xml del proyecto con la sección reporting.

Informe PMD 1: antes de corregir

Con `mvn clean site` se generó `target/site/pmd.html`. El informe reportó 15 violaciones. La diferencia con las 14 de Eclipse se debe a que Maven usa PMD 6.55.0 y el plugin de Eclipse usa PMD 7.28.0. Ambos coinciden en las 10 violaciones de `Email.java`; la diferencia está en `EmailApp.java`, donde Maven reporta 5 y Eclipse 4, porque algunas reglas cambiaron su comportamiento entre PMD 6 y PMD 7.

Regla	Prioridad	Violaciones
FieldNameConventions	1	2
SystemPrintln	2	5
ImmutableField	3	3
UseLocaleWithCaseConversions	3	2
SwitchStmtsShouldHaveDefault	3	1
UseUtilityClass	3	1
ShortVariable	3	1
Total		15

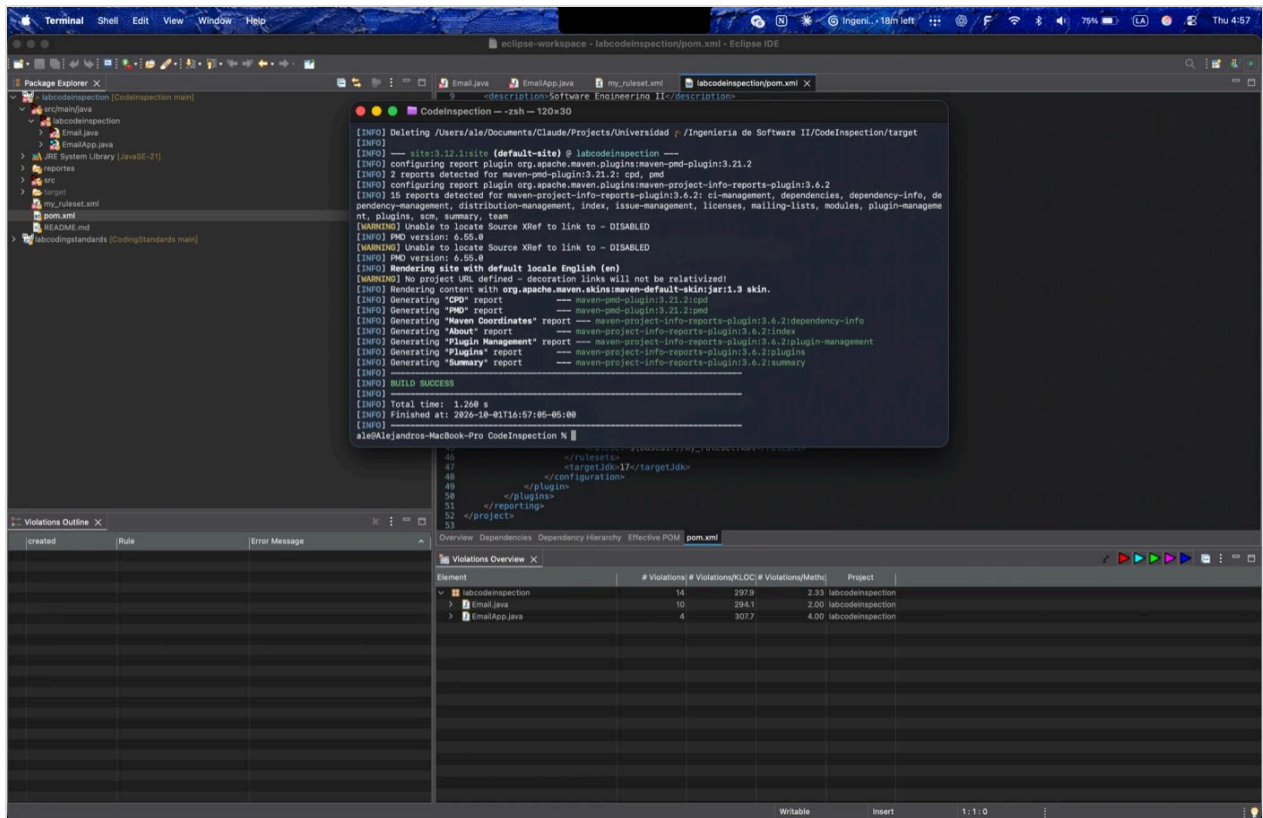


Figura 12. Ejecución de mvn clean site.

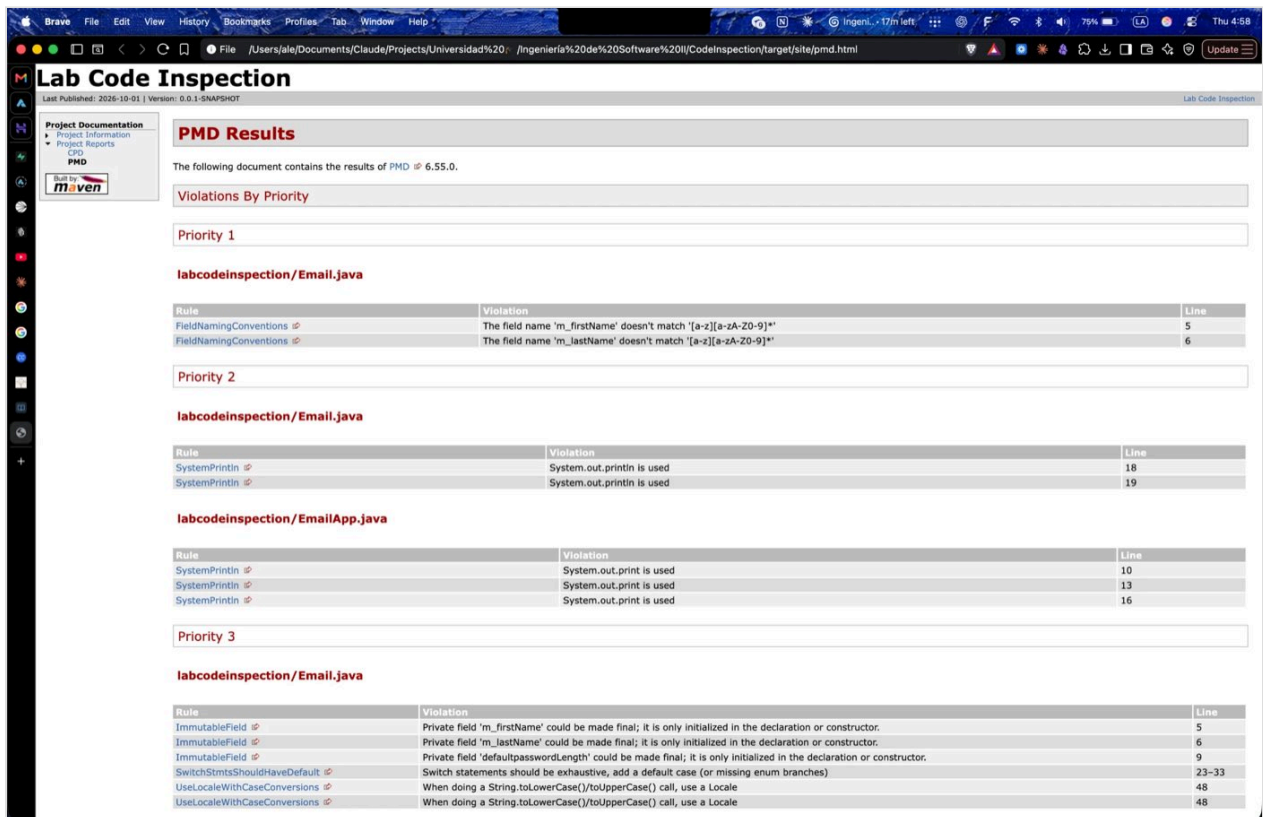


Figura 13. Informe PMD 1: violaciones por prioridad.

SystemPrintln	System.out.print is used	13	
SystemPrintln	System.out.print is used	16	
Priority 3			
labcodeinspection/Email.java			
Rule	Violation	Priority	Line
ImmutableField	Private field 'm_firstName' could be made final; it is only initialized in the declaration or constructor.	3	5
ImmutableField	Private field 'm_lastName' could be made final; it is only initialized in the declaration or constructor.	3	6
ImmutableField	Private field 'defaultpassword.length' could be made final; it is only initialized in the declaration or constructor.	3	9
SwitchStmtsShouldHaveDefault	Switch statements should be exhaustive, add a default case (or missing enum branches)	3	23-33
UseLocaleWithCaseConversions	When doing a String.toLowerCase()/toUpperCase() call, use a Locale	3	48
UseLocaleWithCaseConversions	When doing a String.toLowerCase()/toUpperCase() call, use a Locale	3	48
labcodeinspection/EmailApp.java			
Rule	Violation	Priority	Line
UseUtilityClass	All methods are static. Consider using a utility class instead. Alternatively, you could add a private constructor or make the class abstract to silence this warning.	3	5-26
ShortVariable	Avoid variables with short names like sc	3	8
Files			
labcodeinspection/Email.java			
Rule	Violation	Priority	Line
ImmutableField	Private field 'm_firstName' could be made final; it is only initialized in the declaration or constructor.	3	5
FieldNamingConventions	The field name 'm_firstName' doesn't match '[a-z][a-zA-Z0-9]*'	1	5
ImmutableField	Private field 'm_lastName' could be made final; it is only initialized in the declaration or constructor.	3	6
FieldNamingConventions	The field name 'm_lastName' doesn't match '[a-z][a-zA-Z0-9]*'	1	6
ImmutableField	Private field 'defaultpassword.length' could be made final; it is only initialized in the declaration or constructor.	3	9
SystemPrintln	System.out.println is used	2	18
SystemPrintln	System.out.println is used	2	19
SwitchStmtsShouldHaveDefault	Switch statements should be exhaustive, add a default case (or missing enum branches)	3	23-33
UseLocaleWithCaseConversions	When doing a String.toLowerCase()/toUpperCase() call, use a Locale	3	48
UseLocaleWithCaseConversions	When doing a String.toLowerCase()/toUpperCase() call, use a Locale	3	48
labcodeinspection/EmailApp.java			
Rule	Violation	Priority	Line
UseUtilityClass	All methods are static. Consider using a utility class instead. Alternatively, you could add a private constructor or make the class abstract to silence this warning.	3	5-26
ShortVariable	Avoid variables with short names like sc	3	8
SystemPrintln	System.out.print is used	2	10
SystemPrintln	System.out.print is used	2	13
SystemPrintln	System.out.print is used	2	16

Figura 14. Informe PMD 1: violaciones por archivo.

9. Corrección del código

Regla	Corrección
FieldNamingConventions	m_firstName y m_lastName pasaron a firstName y lastName.
ImmutableField	firstName, lastName y defaultPasswordLength se declararon final.
SwitchStmtsShouldHaveDefault	Se agregó un caso default que asigna el departamento unknown.
UseLocaleWithCaseConversions	toLowerCase() pasó a toLowerCase(Locale.ROOT).
UseUtilityClass	EmailApp se declaró final con constructor privado.
ShortVariable	La variable sc pasó a llamarse scanner.
SystemPrintln	Se mantiene la salida por consola porque es una aplicación de línea de comandos. Se suprimió con @SuppressWarnings("PMD.SystemPrintln"), siguiendo la documentación de supresión de PMD. El informe lista estas supresiones aparte.

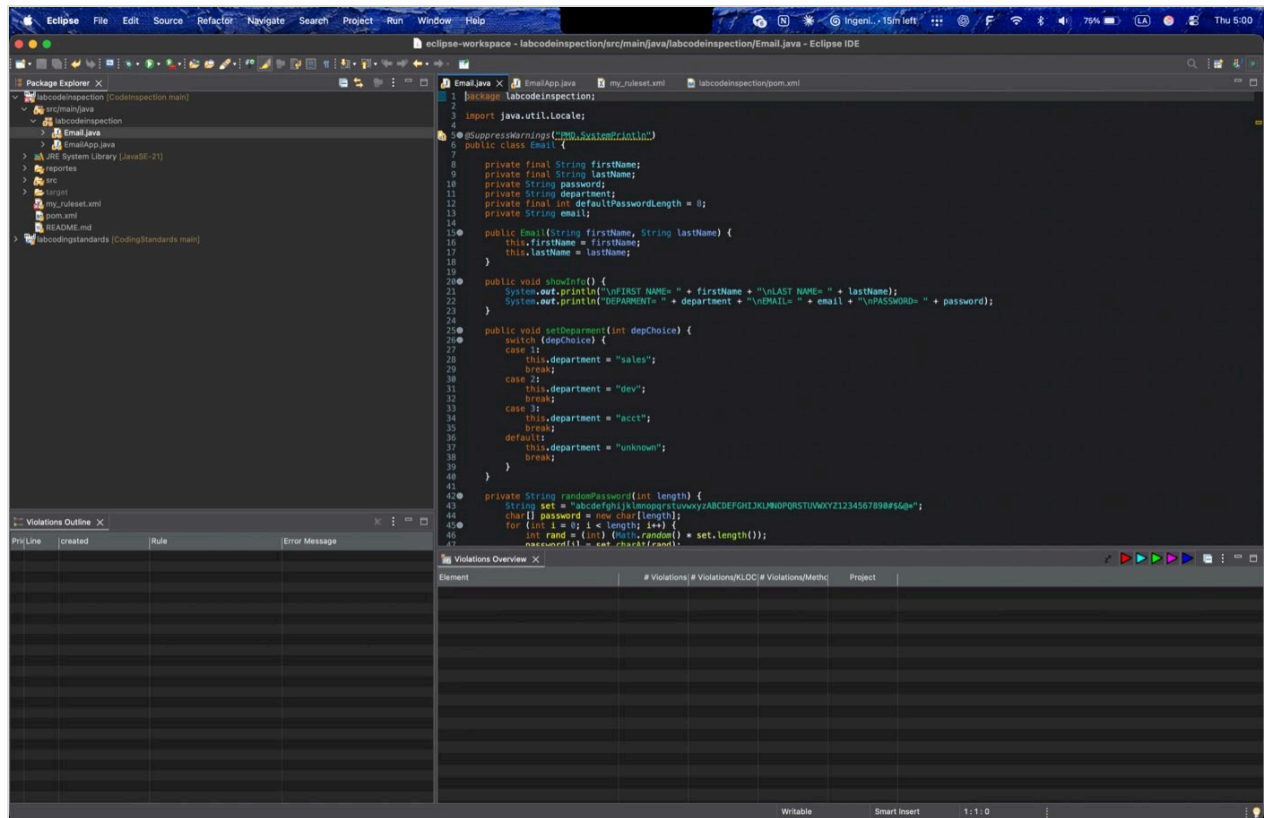


Figura 15. Email.java corregido, sin violaciones en Eclipse.

10. Informe PMD 2: después de corregir

Se volvió a ejecutar `mvn clean site`. El segundo informe indica que PMD no encontró problemas en el código. Solo aparecen las cinco salidas por consola, en la sección Suppressed Violations, marcadas como suprimidas por anotación.

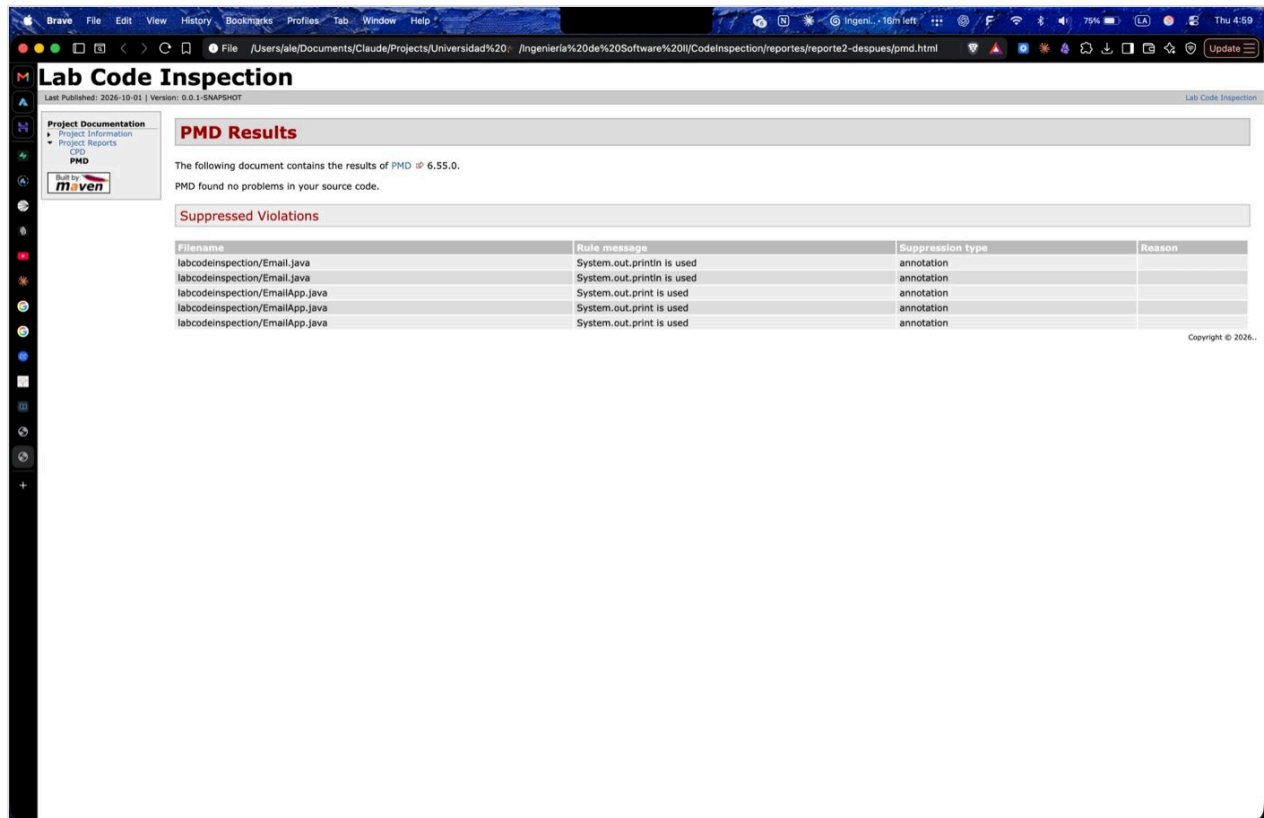


Figura 16. Informe PMD 2: sin problemas en el código.

11. Repositorio

El código, el conjunto de reglas, el `pom.xml` y los dos informes PMD (en la carpeta reportes) están en: <https://github.com/alejandrotapia20/CodeInspection>

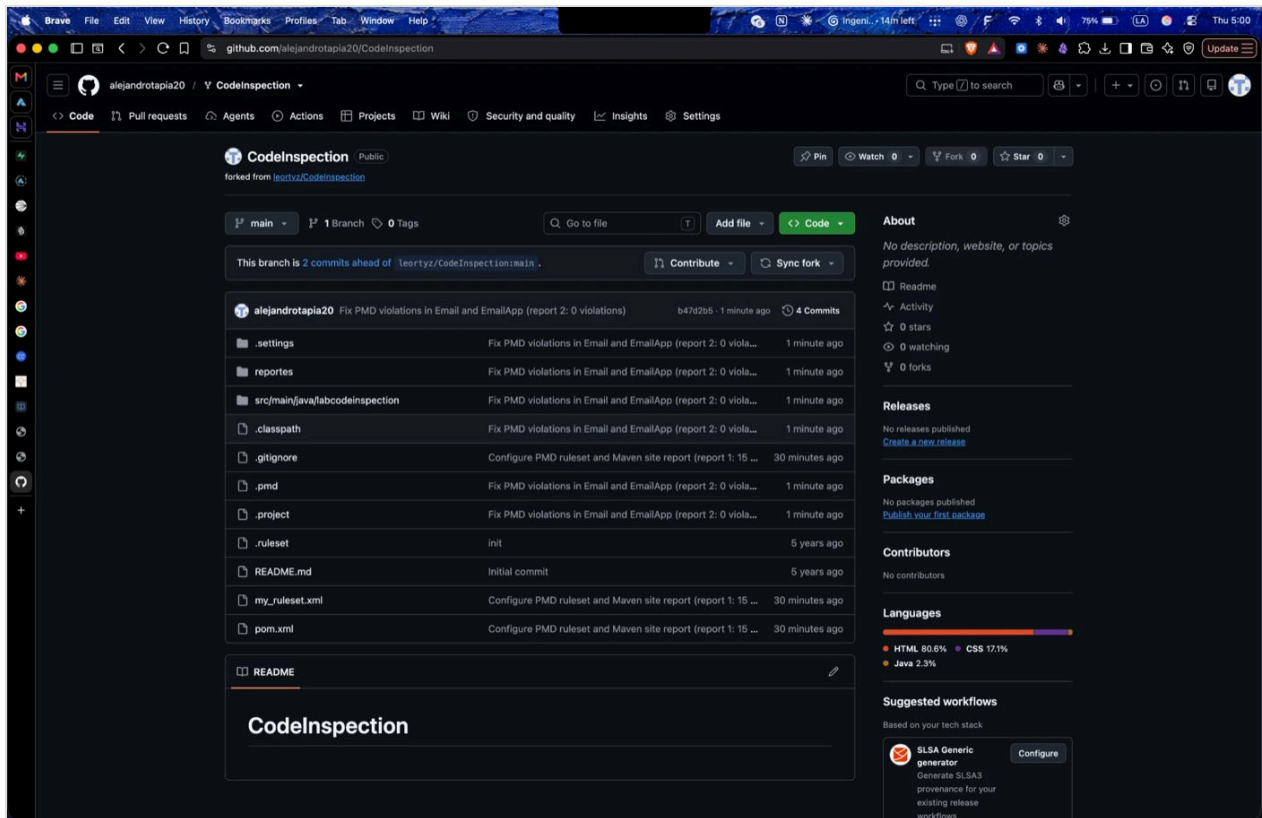


Figura 17. Repositorio en GitHub con los cambios de la práctica.

12. Conclusiones

PMD encontró en un código pequeño problemas de nombres, diseño y portabilidad que no impiden compilar pero sí afectan la mantenibilidad, como atributos que podían ser inmutables o conversiones de texto que dependen del idioma del sistema. Usarlo en Eclipse permite corregir mientras se programa, y usarlo en Maven deja un informe reproducible para todo el equipo. También se vio que la versión del motor importa: el plugin de Eclipse y el de Maven usan versiones distintas de PMD y no cuentan exactamente igual, por lo que conviene fijar la versión en el proyecto. Finalmente, no toda advertencia debe eliminarse cambiando el código: en una aplicación de consola la salida por pantalla es intencional, y lo correcto es suprimirla de forma explícita y documentada.